Information Visualization

Week 3: Theory & Color Spaces

Raz Saremi, Ph.D.
Tandon School of Engineering, NYU

Learning Objectives

By the end of this lecture, you will be able to:

- Explain how the human visual system perceives and processes color
- Compare hardware-centric (RGB) vs. human-centric (LAB, HCL) color spaces
- Convert between color spaces programmatically in Python/JS
- Apply perceptual principles to choose effective color palettes
- Identify and critique poor color choices in real-world visualizations
- Design accessible visualizations for color-deficient viewers

Physics of Light: The Electromagnetic Spectrum

Visible light is a tiny slice of the EM spectrum (380–700 nm). What we call "color" is our brain's interpretation of different wavelengths. Shorter wavelengths appear blue/violet; longer appear red.

Discussion: Why can't displays show all colors we can see?

Key Facts

380 nm	Violet
520 nm	Green
700 nm	Red
Speed	3×10^{8} m/s

Physiology of the Eye: Rods vs. Cones

Rods

- ~120 million in each eye
- Sensitive to low light (scotopic vision)
- No color information — grayscale only
- High sensitivity, low acuity
- Dominant in peripheral vision

Cones (L, M, S)

- ~6 million in each eye
- Require bright light (photopic)
- Three types: L (red), M (green), S (blue)
- Concentrated in the fovea
- Responsible for all color vision

Note: The 20:1 ratio of rods to cones explains why we lose color vision in dim light.

Spectral Sensitivity Curves

Key Insights

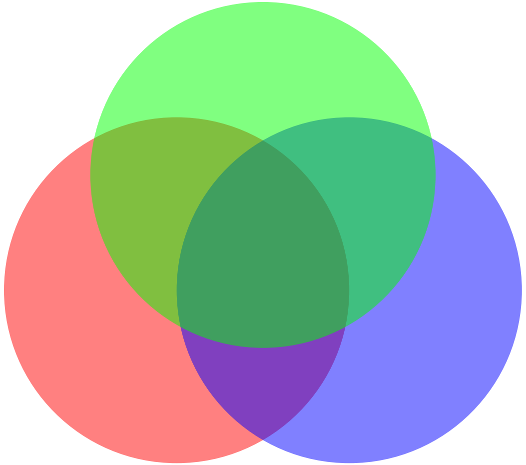
- L and M cones overlap significantly — why red-green deficiency is common
- S cones peak at ~440nm (blue) and are fewest in number
- Most sensitive to green (~555nm) — green dominates luminance

Exercise: Why does Y = 0.2126R + 0.7152G + 0.0722B weight green so heavily?

Trichromatic Theory: The Basis for RGB

Young-Helmholtz Theory (1802/1852)

Any color can be matched by mixing three primary lights (R, G, B) because the eye has exactly three types of cone receptors. RGB displays are engineered to exploit this biological fact.



Additive Mixing

Engineering Implication

Every pixel = 3 sub-pixels (R, G, B). 256^3 = 16,777,216 colors in 24-bit.

Opponent Process Theory

Ewald Hering's theory (1892): We never see "reddish green" or "yellowish blue" because the brain processes color in opposing channels.

Red ↔ Green

Redness vs. Greenness

Blue ↔ Yellow

Blueness vs. Yellowness

Black ↔ White

Luminance channel

Why This Matters

The LAB color space is directly modeled on opponent channels: L* = lightness, a* = red/green, b* = blue/yellow.

Perceptual Non-linearity

Humans are not linear sensors. We're far more sensitive to changes in dark tones than bright tones — why gamma correction exists and naively interpolating RGB produces poor gradients.

Weber-Fechner Law

$S = k \cdot \ln(I)$

Perception is proportional to the logarithm of intensity.

Perceived even ramp (non-linear intensity):

Linear intensity ramp (appears too bright in middle):

Discussion: Which ramp appears to have more evenly spaced steps?

Luminance vs. Brightness vs. Lightness

Luminance (Y)

Physical measure (cd/m^2). Objective, measurable. Linear in physics, not perception.

Brightness

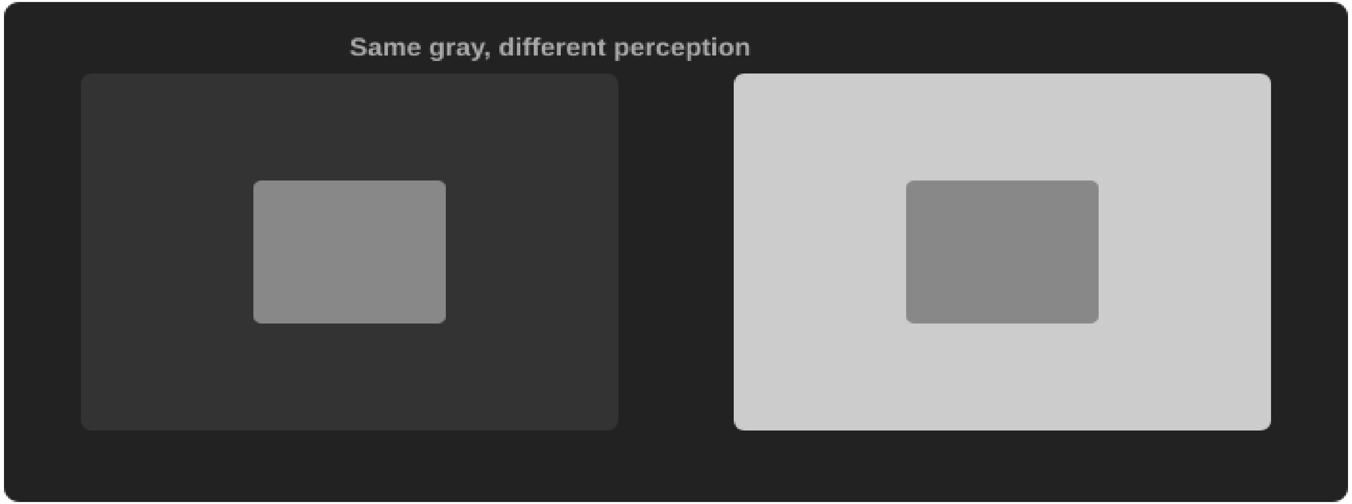
Subjective, absolute perception of light emitted. Not relative. Used informally.

Lightness (L*)

Relative perception normalized against white. Used in LAB/HCL. Perceptually uniform — use this for viz.

Engineering Rule: Always use Lightness (L*) from LAB when creating sequential colormaps.

Simultaneous Contrast



Visualization Impact

The same data color appears different depending on surrounding elements. Always test palettes against actual backgrounds.

Chromatic Adaptation: White Balancing in the Brain

Your brain automatically adjusts its "white point" based on ambient lighting. White paper looks white under both warm and cool lighting — yet the physical wavelengths are completely different.

Industry: Medical Imaging

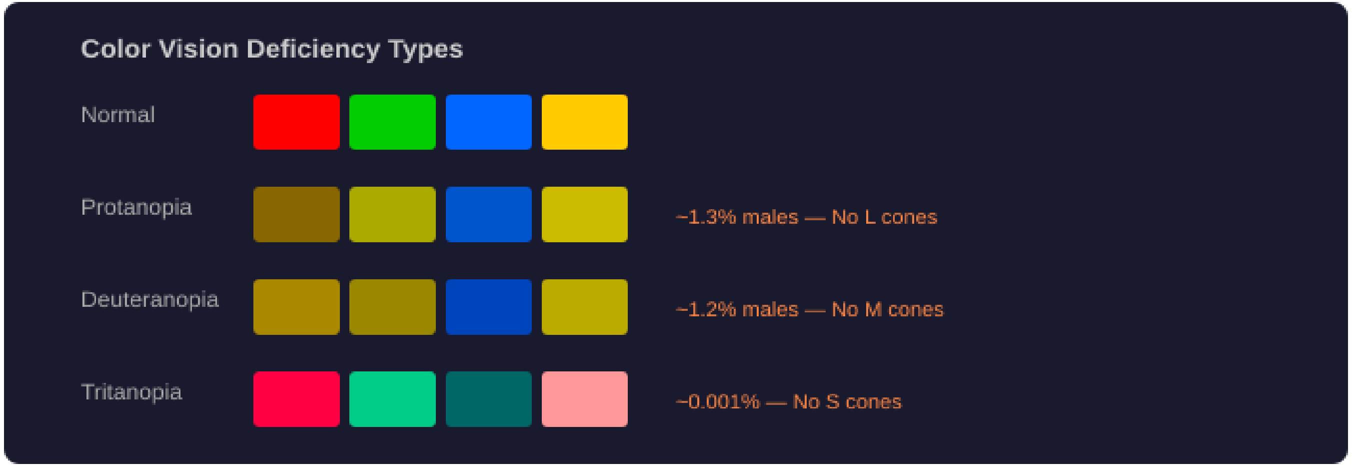
Radiologists use calibrated monitors to avoid adaptation artifacts that could lead to misdiagnosis.

"White" under warm light (3000K)

"White" under cool light (7000K)

Reading: Modeling Color Difference for Visualization Design.
Homework: Explain how ambient lighting affects data interpretation.

Color Deficiency (Color Blindness)



~8% of males and ~0.5% of females have some form of color vision deficiency. In a group 100 people, potentially 4–5 cannot distinguish red from green as you do.

Designing for Accessibility: Safe Palettes

✓ DO

- Use Blue + Orange instead of Red + Green
- Redundant encoding: color + shape/pattern
- Vary lightness, not just hue
- Test with Coblis, Sim Daltonism
- Use ColorBrewer 'colorblind safe' palettes

✗ DON'T

- Rely solely on red/green for meaning
- Use saturated colors differing only in hue
- Assume all users see colors the same
- Use rainbow colormaps for quantitative data
- Ignore WCAG contrast requirements

Optical Illusions: The Stroop Effect

Interactive: Try reading the ink colors aloud! The Stroop Effect (1935) shows that semantic processing conflicts with visual processing — a critical lesson for UI labeling.

Summary: Perception Constraints for Engineers

- Color is a construct of the brain, not a property of light
- 3 cone types → RGB; opponent channels → LAB axes
- Perception is logarithmic → gamma correction matters
- Context changes appearance → test against actual backgrounds
- ~8% of males & 0.5% of females color deficient → always use accessible palettes
- Semantic processing can override visual → the Stroop Effect

Part 2

Color Spaces & Models

The math and engineering behind color representation

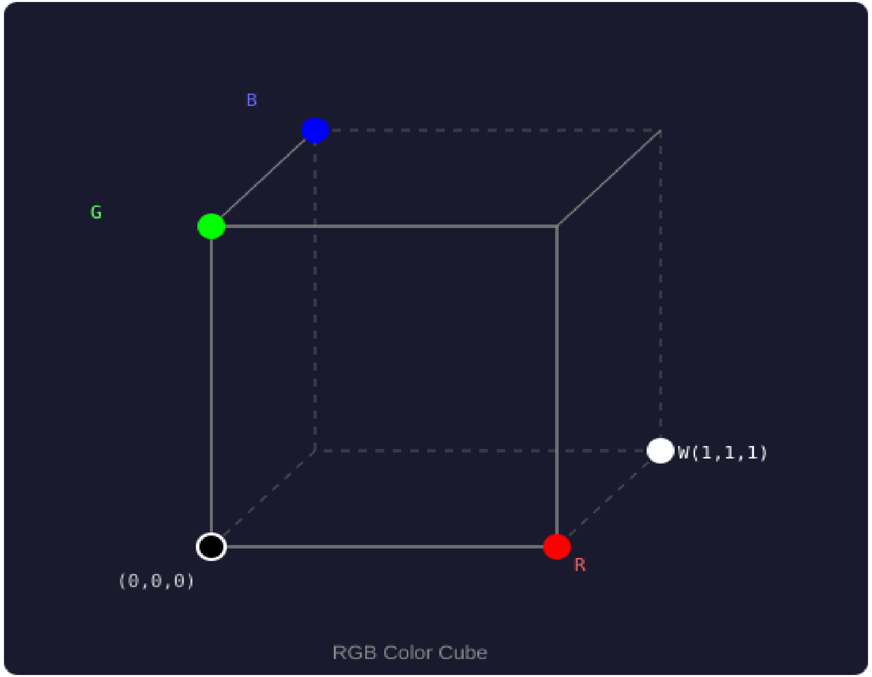
RGB (Red, Green, Blue): The Additive Model

RGB is an additive color model: all three at full intensity = white.

Native language of every digital display.

Properties

- 3 channels: R, G, B ∈ [0, 255]
- Geometry: unit cube in 3D space
- Black = (0,0,0), White = (255,255,255)
- Device-dependent: same values look different across monitors
- sRGB is the standard gamut for web



RGB Cube Geometry: Coordinate Systems in Code

Every color is a point in 3D space. RGB distance is Euclidean but does NOT correspond to perceived difference.

```
# RGB distance — NOT perceptually meaningful
import numpy as np
color_a = np.array([255, 100, 50])   # orange
color_b = np.array([50, 100, 255])    # blue
rgb_dist = np.linalg.norm(color_a - color_b)
# ≈ 290.0 — but is this "big"? RGB can't tell you.
```

We need a perceptually uniform space — that's LAB.

Limitations of RGB

Unintuitive	What's (180, 50, 220)? It's purple — impossible to predict.
Non-uniform	Equal RGB steps ≠ equal perceived steps.
Device-dependent	(200, 50, 50) looks different on laptop vs phone.
No hue separation	Making colors 'more vivid' requires changing all 3 channels.

Color pickers use HSV/HSL; visualization engineers use LAB/HCL.

HSV/HSL: Cylindrical Color Models

HSV/HSL rearrange the RGB cube into an intuitive cylindrical shape:

Hue (H)

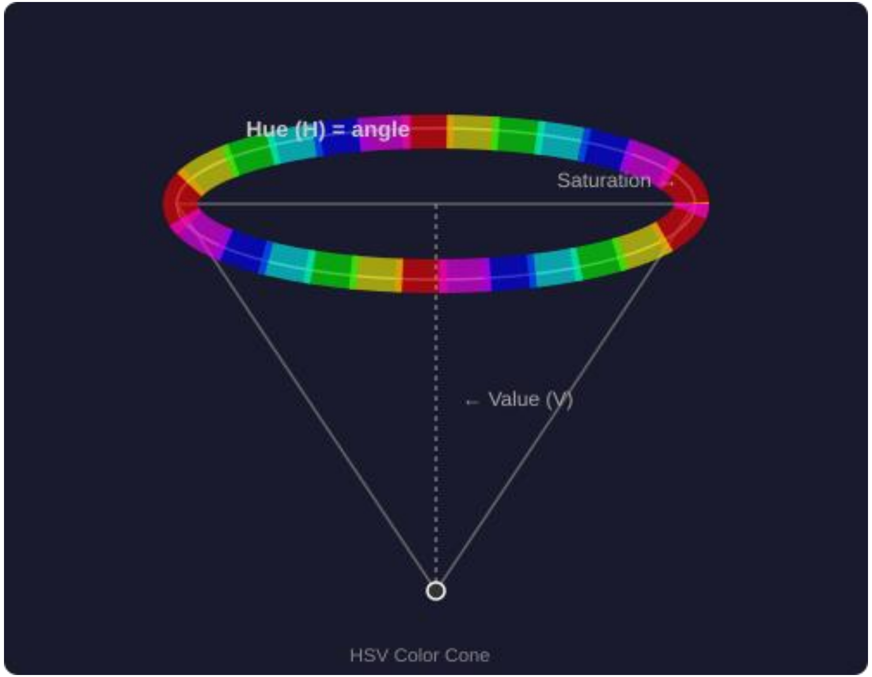
Angle: 0°=Red, 120°=Green, 240°=Blue

Saturation (S)

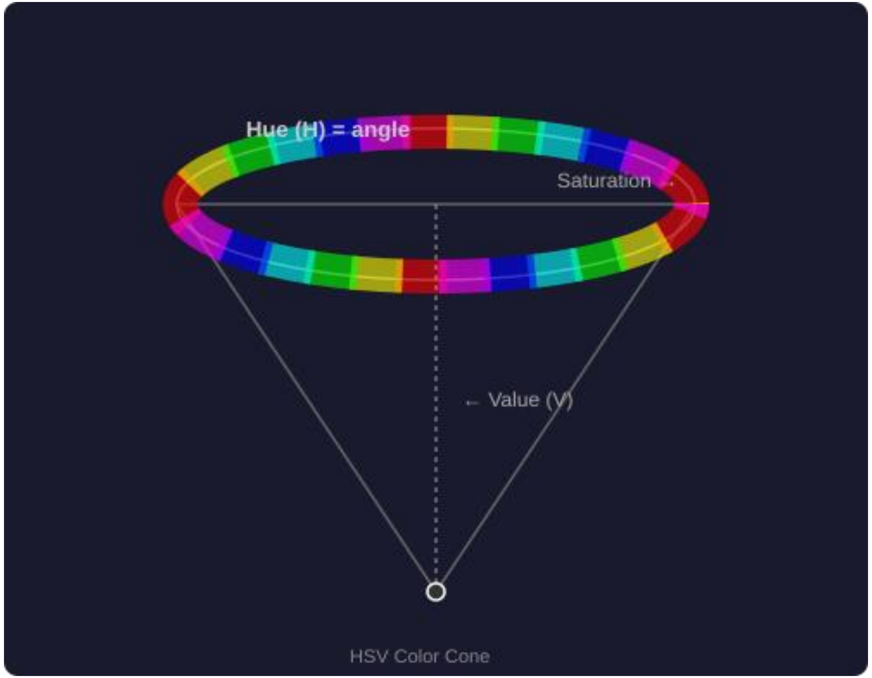
Vividness: 0=gray, 1=fully saturated

Value/Lightness

V: brightness (0=black); L: midpoint-adjusted



Visualizing the HSV Cone/Cylinder



Geometric Structure

HSV = cone: top = saturated colors, apex = black.

HSL = bicone: black at bottom, white at top, saturated at equator.

CSS: hsl(210, 80%, 50%) = vivid blue.

Exercise: In CSS, create 5 shades of blue using HSL by varying only L.

5 shades of blue using HSL by varying only L

Component	Value	Description
Hue (H)	240	Position on the color wheel (0-360). 240 is Blue.
Saturation (S)	100%	Intensity of the color. 100% is full color; 0% is grayscale.
Lightness (L)	Varies	The amount of white/black. **0% is black**, **100% is white**.

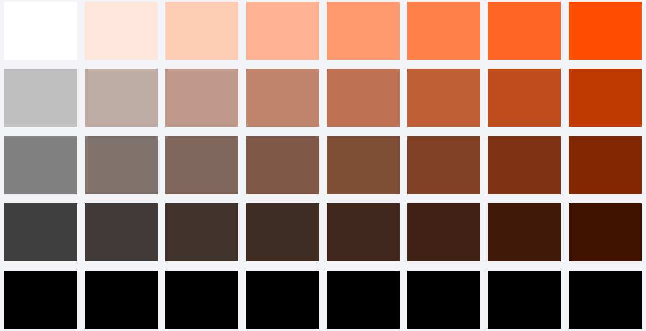
Constant H: 240, Constant S: 100% | Varying L

1. .blue-very-dark { background-color: hsl(240, 100%, 10%);}
2. .blue-dark { background-color: hsl(240, 100%, 30%);}
3. .blue-medium { background-color: hsl(240, 100%, 50%); /* Pure Blue */}
4. .blue-light { background-color: hsl(240, 100%, 70%);}
5. .blue-very-light { background-color: hsl(240, 100%, 90%); }

Advantage of HSV: Intuitive for User Interfaces

HSV maps to how humans think: "What color?" (H), "How vivid?" (S), "How bright?" (V).

Color Picker Grid (SV plane)

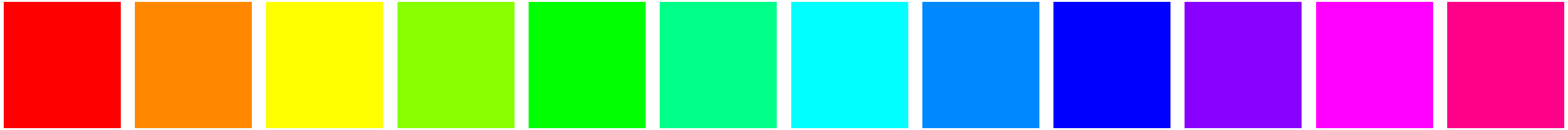


Why it works

- Users think in hue/vividness/brightness
- Easy harmonious palettes: rotate hue
- CSS natively supports hsl()
- Perfect for interactive color selection

Disadvantage: HSV is NOT Perceptually Uniform

Equal steps in H, S, or V do NOT produce equal perceived differences. Hue sweep at S=100%, V=100%:



← Yellow appears much brighter than blue at same S,V →

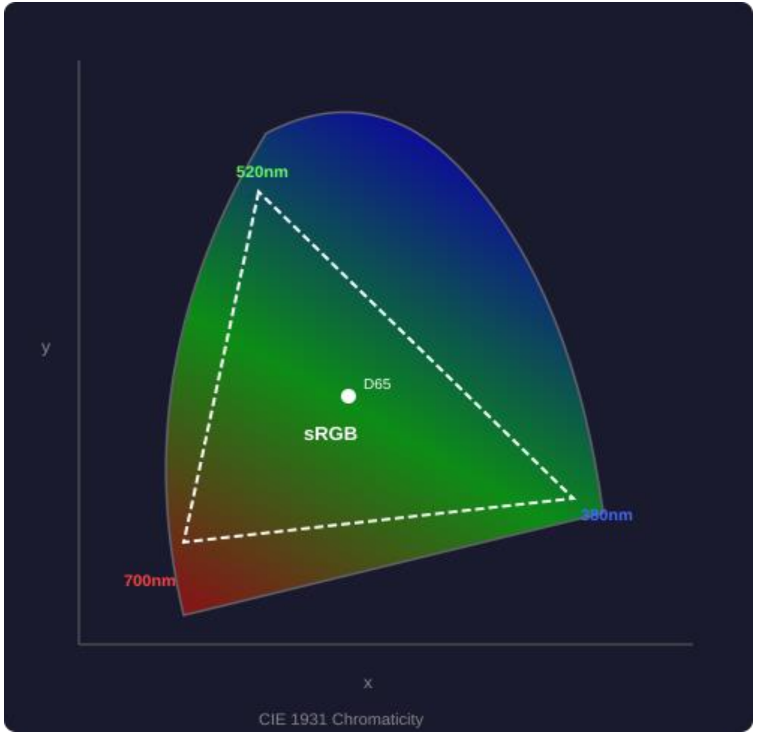
Rule: Use HSV for UI color picking. Use LAB/HCL for data encoding. Never build colormaps by sweeping HSV hue.

CIE XYZ: The Foundation of Color Science (1931)

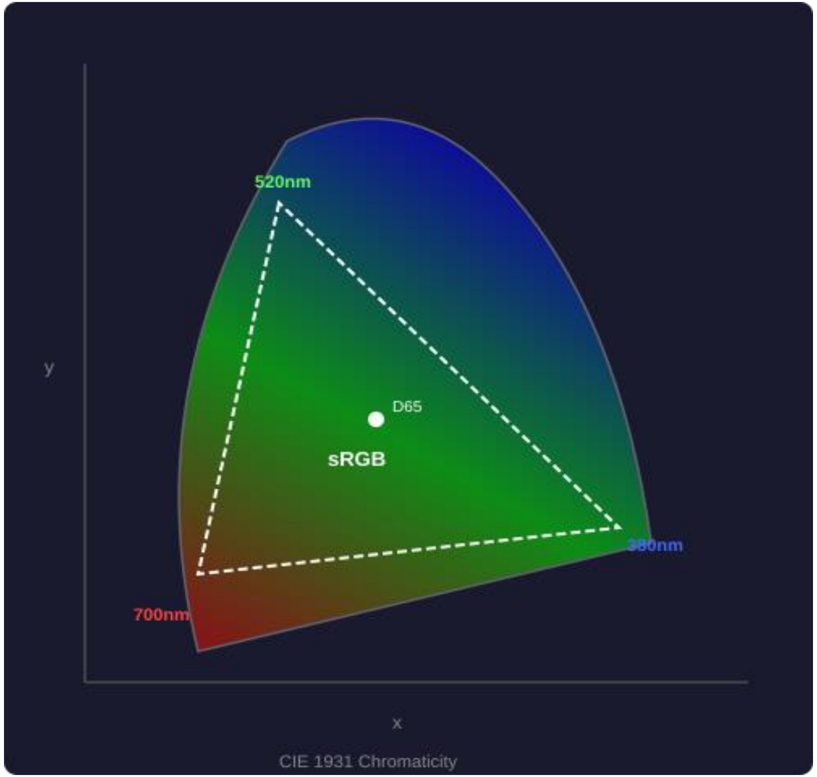
CIE 1931 XYZ is the mathematical foundation of all modern color science, created using psychophysical experiments with human observers.

Key properties:

- Y channel = luminance (human brightness)
- All visible colors have non-negative XYZ
- Device-independent: the 'Rosetta Stone'
- RGB → XYZ → LAB conversion pipeline



The Chromaticity Diagram: Gamut Visualization

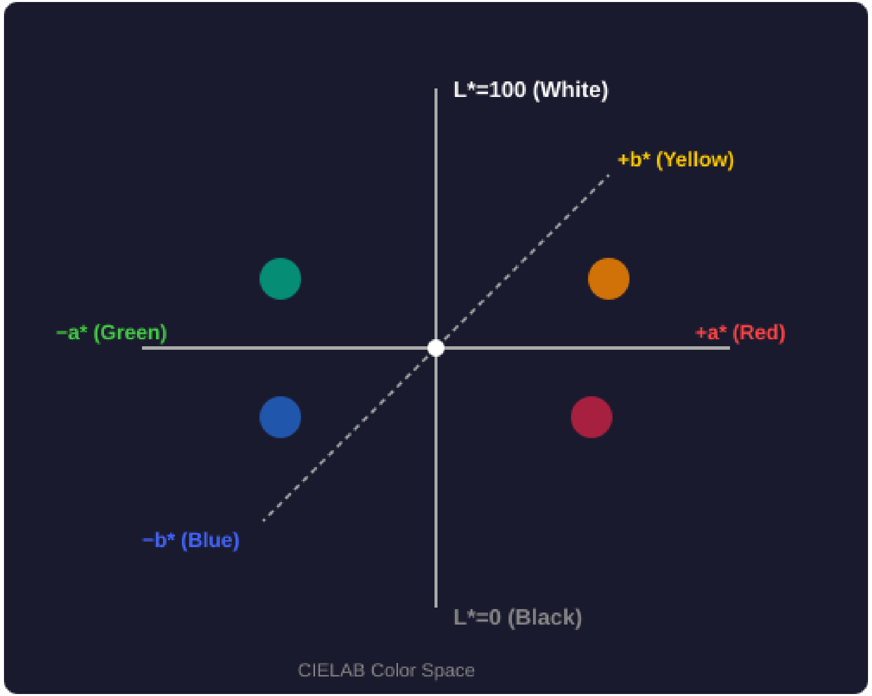


Reading the Diagram

- Horseshoe = all visible colors
- Curved edge = spectral colors
- Straight bottom = 'line of purples'
- D65 white point at center
- sRGB triangle = your monitor's gamut
- Display P3 is ~25% larger
- Outside triangle = visible but not displayable

CIELAB (LAB): The Perceptually Uniform Space

CIELAB (1976): a given numerical change corresponds to a similar perceived change regardless of position in the space.



Why LAB Matters

- Euclidean distance ≈ perceived difference
- L* matches human lightness perception
- a*, b* match opponent process channels
- Colormaps built in LAB look smoother

"If you only learn one color space, learn LAB."

Understanding the Axes: L*, a*, b*

L* (Lightness)

`Range: 0 (black) → 100 (white)`

Matches human lightness. ΔL*=10 is equally noticeable across the range.

a* (Green ↔ Red)

`Range: −128 (green) → +127 (red)`

Models the Red-Green opponent channel. Zero = neutral gray.

b* (Blue ↔ Yellow)

`Range: −128 (blue) → +127 (yellow)`

Models the Blue-Yellow opponent channel. Defines chromatic content.

Delta E (ΔE): Perceived Color Difference

ΔE = Euclidean distance in LAB space. Quantifies how different two colors look.

$$\Delta E = \sqrt{(L_1^*-L_2^*)^2 + (a_1^*-a_2^*)^2 + (b_1^*-b_2^*)^2}$$

`< 1.0`	Not perceptible
`1–2`	Barely perceptible
`2–10`	Noticeable at a glance
`11–49`	Clearly different
`> 100`	Opposite colors

For viz: aim for ΔE > 20 between categorical colors.

HCL (Hue-Chroma-Luminance): Gold Standard for Viz

★ HCL = polar form of LAB. Combines LAB's perceptual uniformity with HSV's intuitive hue reasoning.

$H = \operatorname{atan2}(b^*, a^*)$	$C = \sqrt{(a^{*2} + b^{*2})}$	$L = L^*$
Hue angle (0-360°)	Chroma (saturation)	Luminance

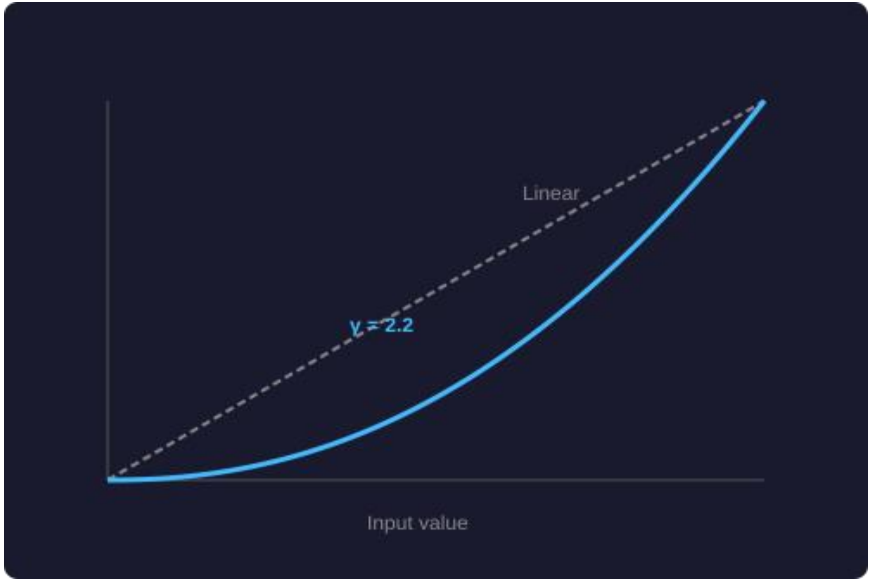
- Constant L = constant perceived brightness → fair categorical comparisons
- Varying H at constant C, L → equally distinct categories
- Varying L at constant H, C → perceptually uniform sequential scales

Comparison: RGB vs. HSV vs. LAB

Property	RGB	HSV/HSL	CIELAB
Perceptually Uniform	✗ No	✗ No	✓ Yes
Intuitive for Humans	✗ Poor	✓ Good	~ Moderate
Device Independent	✗ No	✗ No	✓ Yes
Best for Displays	✓ Native	~ Derived	✗ Convert
Best for Colormaps	✗ No	✗ No	✓ Yes
Distance = Perception	✗ No	✗ No	✓ ≈ Yes

Use RGB for storage, HSV for UI, LAB/HCL for visualization.

Gamma Correction: 50% Gray ≠ (128,128,128)



What is Gamma?

V_out = V_in^γ (γ ≈ 2.2 for sRGB)

128/255 ≈ 0.5 sRGB = only ~21.8% luminance.

BT.709 grayscale:

Y = 0.2126R + 0.7152G + 0.0722B

(Apply to linear RGB, not sRGB!)

Exercise 2: Python — convert image to 3 grayscale versions (naive avg, BT.709, LAB L-channel). Compare histograms.

Gamma-Correct Grayscale: BT.709 Pipeline

1 Linearize (Undo γ)

`V_linear = V_sRGB^2.2`

Convert display values → physical light intensity

▼

2 Apply BT.709 Weights

`Y = 0.2126R + 0.7152G + 0.0722B`

Weight channels by human spectral sensitivity

▼

3 Re-apply Gamma

`V_display = Y_linear^(1/2.2)`

Compress back for monitor display

Python

```python
import cv2;  import numpy as np
# 1. Linearize (undo sRGB gamma 2.2)
img  = cv2.imread('photo.jpg')
rgb  = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
srgb = rgb.astype(np.float32) / 255.0
lin  = np.power(srgb, 2.2)          # → linear light

# 2. BT.709 weighted sum in linear space
Y = (0.2126 * lin[:,:,0] +
     0.7152 * lin[:,:,1] +
     0.0722 * lin[:,:,2])

# 3. Re-apply gamma for display
gray = np.power(Y, 1/2.2)       # → sRGB out
out  = (gray * 255).astype(np.uint8)
```

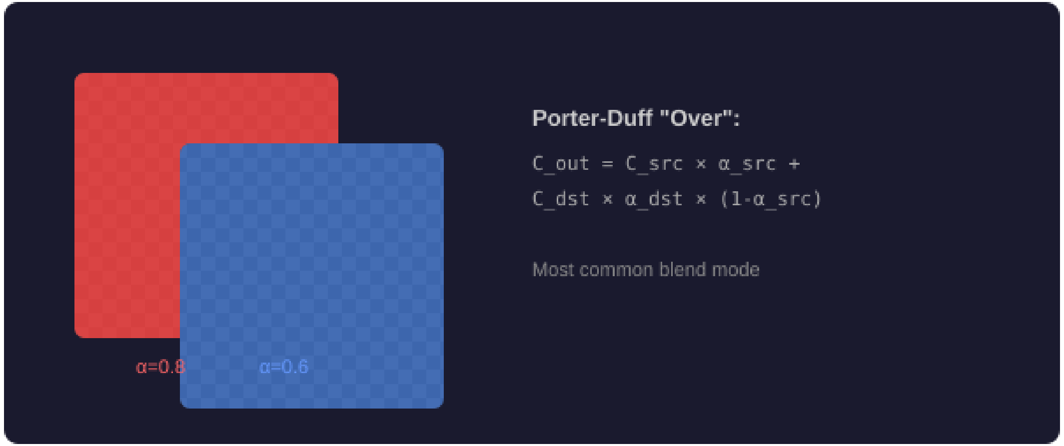
Common Mistake: Averaging RGB directly (R+G+B)/3 skips linearization and ignores spectral weights — producing incorrect luminance, muddy midtones, and artifacts in gradients.

Gamut Mapping: When a Color Can't Be Displayed

When a color falls outside a display's gamut, it must be mapped to the nearest displayable color.

Clipping	Cap out-of-range values. Fast but can shift hue.
Perceptual	Compress entire gamut proportionally. Preserves relationships.
Relative Colorimetric	Map white point, clip rest. Good for photos.
Absolute Colorimetric	Exact reproduction where possible. Used for proofing.

Alpha Compositing: Porter-Duff Operators



Porter-Duff (1984)

12 compositing ops for semi-transparent layers.

CSS: rgba(255,0,0,0.5)

SVG: opacity="0.5"

GL: glBlendFunc()

In viz, alpha is used for overplotting (scatter), layered maps, selection highlighting.

Order of layers matters!

From Web-Safe Colors to HDR

1996: Web-Safe

216 colors (6×6×6). For 8-bit displays.

2000s: 24-bit sRGB

16.7M colors. Current web standard. ~35% visible gamut.

2020s: Display P3 / HDR

Wider gamut + HDR. CSS: color(display-p3 1 0 0).

CSS Color Level 4 supports lab(), lch(), color() functions natively.

Code: Converting RGB to LAB in Python

```python
from skimage import color
import numpy as np

# Convert sRGB to LAB
rgb = np.array([[[255, 100, 50]]], dtype=np.uint8)
lab = color.rgb2lab(rgb)
L, a, b = lab[0, 0]  # L≈58, a≈43, b≈55

# Delta E between two colors
c1 = np.array([58, 43, 55])
c2 = np.array([32, 80, -108])
delta_e = np.linalg.norm(c1 - c2)  # ≈ 170.4

# JS: import {lab} from 'd3-color';
```

Libraries: scikit-image, colour-science (Python); d3-color, chroma.js (JS)

Part 3

Color Usage in Visualization

Applying theory to make data tell truthful stories

Qualitative Schemes (Categorical Data)

Qualitative palettes use maximally distinct hues at similar lightness/chroma. No order — just identity.

Design Rules

- Max 7–10 categories
- Vary hue at constant L and C (HCL!)
- Check $\Delta E > 20$ between all pairs

Common Tools

- ColorBrewer 'Set2', 'Dark2'
- d3.schemeCategory10
- Tableau 10 palette

Sequential Schemes (Continuous Data)

Vary lightness monotonically. Encode ordered, continuous data.

Blues

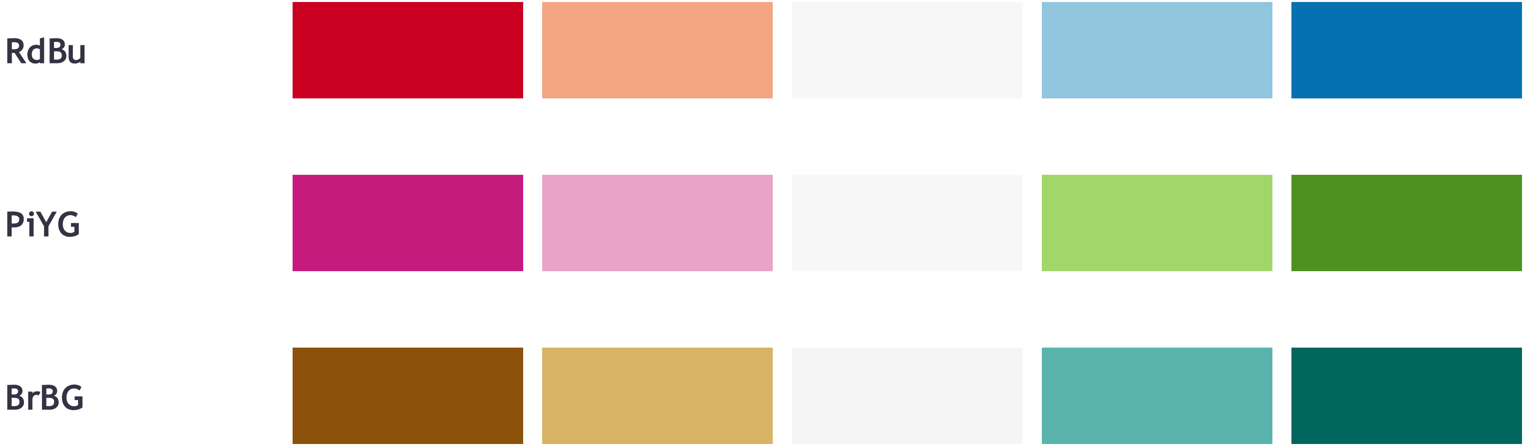
Viridis

Magma

Litmus test: Print in B&W. If ordering is still clear, the palette is good.

Diverging Schemes (Critical Midpoint)

Two sequential ramps from a neutral midpoint. For data with meaningful center (zero, average, threshold).



Finance: Stock heatmaps use diverging palettes. Blue/Orange > Red/Green for accessibility.

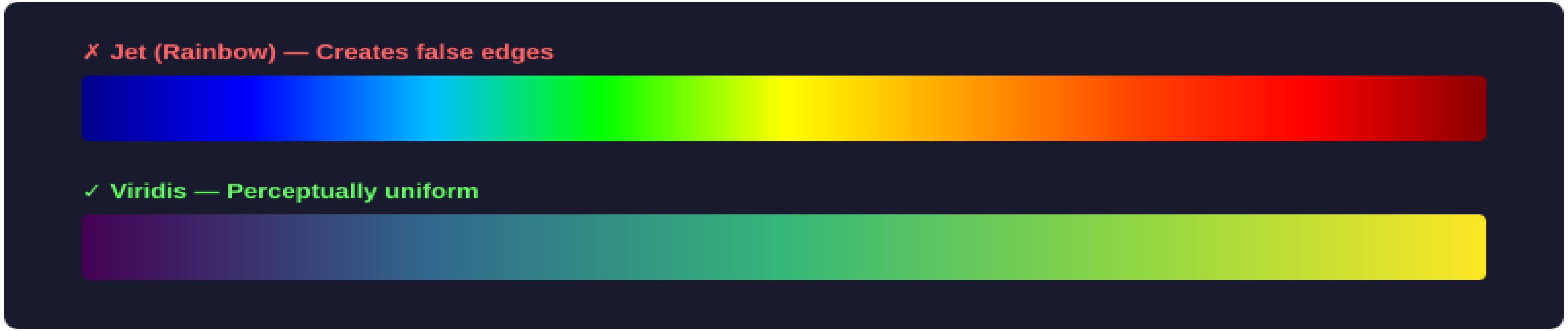
Creating Perceptually Linear Ramps

Uniform ΔE per data step — no false edges, no brightness spikes.

Recipe for a Good Sequential Colormap:

1. Choose start/end colors in HCL
2. Ensure L* is monotonic
3. Keep chroma within displayable gamut at each L*
4. Interpolate in HCL or LAB
5. Verify constant ΔE between consecutive steps (±10%)
6. Convert to sRGB for display
7. Test in grayscale and color blindness simulation

The Rainbow Map Problem: Why 'Jet' is Harmful



Rainbow/Jet creates artificial boundaries at yellow-green and cyan transitions. Scientists see patterns that don't exist.

Climate Science

Jet colormaps in weather data create false 'edge' artifacts in temperature maps.

Viridis & Magma: Designing Scientific Colormaps

Viridis (2015, van der Walt & Smith) — perceptually uniform, colorblind-safe, grayscale-readable.

✓	**Perceptually Uniform**	L* increases monotonically ~15→95
✓	**Colorblind Safe**	Distinguishable under all CVD types
✓	**Grayscale Readable**	Ordering preserved when desaturated
✓	**Print Friendly**	Works on screen and paper
✓	**Aesthetically Smooth**	Purple → teal → yellow transition

```
plt.imshow(data, cmap='viridis')  # Default since matplotlib 2.0
```

Semantic Color Associations

Colors carry cultural meaning. Leverage associations but be aware of variation.

	Red	Danger, Stop, Loss, Hot	Luck, Prosperity (China)
	Green	Go, Growth, Nature, Profit	Similar globally
	Blue	Trust, Cold, Calm, Corporate	Immortality (China)
	Yellow	Warning, Caution, Optimism	Royalty, Power
	Gray	Neutral, Disabled, N/A	Maturity, Stability

Data Density: When Color Becomes Noise

As density increases, individual colors blur. Consider alternatives beyond ~7 categories.

Small Multiples	Split by category, single color per panel.
Opacity/Alpha	Transparency shows density. Darker = more points.
Binning/Heatmaps	Aggregate into bins, map count to sequential color.
Highlight Only	Gray everything; color only the focus set.

Interaction: Color for Selection & Brushing

Interactive Visualization Patterns:

Brushing: Click/drag to select → highlight in accent, gray out rest.

Linking: Selection in one view changes all linked views.

Focus+Context: Vivid for focus, muted for context.

The Stroop Effect (above) shows why consistent color semantics matter.

Backgrounds & Text: Maximizing Contrast

WCAG 2.1 requires minimum contrast ratios for accessibility.

AA: 4.5:1 ratio (normal text)

AAA: 7:1 ratio (enhanced)

✗ Yellow on white: 1.1:1

✗ Dark blue on dark purple

Dark Mode: Use #121212 not #000000 to reduce OLED smearing. Text: #E0E0E0, not pure white.

Tools for Engineers

ColorBrewer 2.0	Gold standard for cartographic palettes.
Viz Palette	Test discriminability and color blindness.
Adobe Color	Harmonious palettes with color theory rules.
Coblis / Sim Daltonism	CVD simulators for your screenshots.
Coolors.co	Quick palette generator with contrast checker.

Also: chroma.js, d3-color, seaborn, ggplot2 scales, Figma plugins

Case Study: Good vs. Bad Color in Dashboards

✗ Poor Choices

- Rainbow on continuous data
- Red + Green for profit/loss
- 12+ categorical colors
- Bright saturated on white bg
- No consistent color mapping

✓ Effective Choices

- Viridis/sequential for continuous
- Blue/Orange diverging (accessible)
- Max 5–7 categorical colors
- Muted colors, gray for context
- Consistent semantics throughout

Homework 2: Find a public dashboard. Identify 3 color choices. Classify and critique each.

Color Review Checklist

Use before publishing any visualization:

- ☐ Color scheme matches data type? (Categorical→Qualitative, Ordered→Sequential, Centered→Diverging)
- ☐ Lightness (L*) monotonic for sequential colormaps?
- ☐ Passes colorblind simulation? (Coblis, Sim Daltonism)
- ☐ Prints legibly in grayscale?
- ☐ Text contrast ≥ 4.5:1 (WCAG AA)?
- ☐ No more than 7 categorical colors without redundant encoding?
- ☐ Color semantics consistent across all charts?
- ☐ Palette tested against actual background?
- ☐ Users can extract values from legend without guessing?

Key Takeaways

1. Color is perception, not physics — design for the human visual system
2. Match space to task: RGB for code, HSV for UI, LAB/HCL for viz
3. Perceptual uniformity prevents false patterns — never use rainbow
4. Accessibility is not optional — ~8% of males are color deficient
5. Validate with tools: ColorBrewer, Viz Palette, Coblis

Ware (2021) Information Visualization • Munzner (2014) Visualization Analysis & Design • colorbrewer2.org • matplotlib.org